

Reprogramming a Frozen LLM for Motor Temperature Forecasting: A Multi-Session, Multi-Seed Reproduction and Baseline Study

Jeevan Merkaji Somanna
Independent Researcher
jeevanms5355@gmail.com

September 10, 2026

Abstract

We reproduce a simplified version of Time-LLM [Jin et al., 2024], which forecasts time series by reprogramming a frozen, pretrained language model rather than training a forecaster from scratch. Using a GPT-2 (124M) backbone [Radford et al., 2019] on CPU-only hardware, we implement the reprogramming mechanism and reversible instance normalization (RevIN) [Kim et al., 2022], and evaluate against two baselines – a from-scratch LSTM and a linear DLinear model [Zeng et al., 2023] – on Kaggle’s real-world Electric Motor Temperature dataset [Kirchgässner et al., 2021a,b]. Unlike a typical single-run comparison, we evaluate across **3 independent real-world recording sessions** (`profile_id` 70, 62, 44) $\times$ **3 random seeds** (9 runs per method, 27 total), reporting mean $\pm$ standard deviation rather than single-point estimates. Our central finding is that **no single method dominates across sessions**: the from-scratch LSTM baseline wins on `profile_id=70` (31% lower MSE than Time-LLM, replicating an earlier single-session result), DLinear and Time-LLM are statistically indistinguishable at a near-zero error floor on `profile_id=62`, and **Time-LLM wins outright on `profile_id=44`** (27% lower MSE than LSTM, 10% lower than DLinear). Averaged across all three sessions, Time-LLM achieves the lowest mean test MSE of the three methods (0.0290 vs. LSTM’s 0.0337 and DLinear’s 0.0348), while LSTM achieves the lowest mean MAE (0.0581 vs. Time-LLM’s 0.0793) – a split verdict that is itself the main result. This directly complicates the common assumption, supported by our own earlier single-session pilot, that a small from-scratch baseline uniformly dominates LLM-reprogramming on data-abundant, single-sensor tasks, and demonstrates why multi-session evaluation is necessary before drawing method-level conclusions from time-series forecasting comparisons – a single session, however carefully chosen, can misrepresent the general ranking of methods on a dataset with heterogeneous recording sessions.

1 Motivation

Forecasting motor temperature from sensor data is a well-studied problem usually tackled with purpose-built sequence models (LSTMs, TCNs, Transformers trained from scratch on the target domain). Time-LLM [Jin et al., 2024] proposes an alternative: keep a large pretrained language model frozen, and train only a lightweight “reprogramming” layer that translates numeric time-series patches into something resembling the LLM’s native token-embedding space. The appeal is that the frozen LLM contributes general sequence-reasoning ability learned from massive pretraining, without the cost of training a large model from scratch or fine-tuning it.

This project asks a practical question relevant to anyone considering this approach for a real sensor-forecasting task: *on modest hardware, with a small frozen backbone, does reprogramming actually beat a simple task-specific baseline you could train in an afternoon?* We answer this empirically rather than assuming the paper’s headline results (obtained with a 7B-parameter backbone and a full training budget) transfer to a constrained, CPU-only, GPT-2-scale setting.

A first pass at this question (single session, single seed) found the from-scratch LSTM baseline winning outright. But a single session/seed comparison risks over-generalizing from what could be session-specific noise or an artifact of one random initialization – a known concern in time-series forecasting benchmarks, where results can be sensitive to both the specific recording window and training stochasticity. This work extends that comparison to **3 independent real-world sessions × 3 seeds per method**, asking: *does the from-scratch baseline’s advantage hold up across multiple independent slices of real data, or was the original single-session result an artifact of that one session?* The answer, reported in Section 4, is that it was partly an artifact: the LSTM’s advantage replicates on the original session but reverses on another, with the third session landing all three methods within noise of each other.

2 Method

Time-LLM’s core mechanism, as implemented here:

1. **Patch embedding.** The input window is split into overlapping patches (`patch_len=16`, `stride=8`) and linearly projected to a small embedding dimension `d_model`.
2. **Patch reprogramming.** A multi-head cross-attention layer lets each patch embedding attend over a fixed subsample of the frozen LLM’s own vocabulary embedding matrix (“text prototypes”), producing an embedding in the LLM’s native space without ever editing the LLM’s weights.
3. **RevIN** [Kim et al., 2022]. Each input window is normalized using its own mean/std (not a global training-set statistic), and the model’s output is denormalized with the same per-window statistics before the loss is computed. This is the mechanism used to handle distribution shift between windows.
4. **Output projection.** The frozen LLM’s output over the patch positions is flattened and linearly projected to the forecast horizon.

Only the patch embedding, reprogramming layer, RevIN’s two affine parameters, and the output projection are trained. The LLM itself never updates – in our runs, 0.1–1.1% of total parameters were trainable, consistent with the paper’s efficiency claims. Prompt-as-Prefix (PaP), the paper’s optional text-conditioning mechanism, is disabled throughout this matrix (`--no_prompt`) to prioritize seed/session coverage within a CPU-only compute budget; it was evaluated separately in an earlier single-session pilot and roughly quintuples per-sample compute.

Baselines.

- **LSTM** – 2-layer, 64 hidden units, same RevIN normalization and windowing code as Time-LLM, so the comparison isolates the forecasting method rather than data-handling differences.
- **DLinear** [Zeng et al., 2023] – a simple linear trend/seasonal decomposition model, included as a fast, near-zero-capacity reference point.

Deviations from the original paper, made for tractability on CPU-only, no-GPU hardware: a GPT-2 (124M) backbone instead of the paper’s primary Llama-7B; smaller `d_model`/prototype counts and shorter training schedules than the paper’s tuned, per-dataset configurations.

3 Experimental Setup

Dataset. Kaggle’s Electric Motor Temperature dataset [Kirchgässner et al., 2021a,b] (Paderborn University): 185 hours of PMSM (permanent magnet synchronous motor) test-bench recordings at 2Hz, 69 independent measurement sessions (`profile_id`) concatenated with no timestamp column. We select 3 sessions to test cross-session robustness:

<code>profile_id</code>	Rows	Notes
70	25,677	mid-sized session, used in the single-session pilot
62	25,600	–
44	26,341	–

Target: `pm` (permanent-magnet/rotor temperature). Window size `seq_len=240/pred_len=60` (2 minutes of history → 30 seconds ahead), chosen to avoid the near-trivial persistence regime of shorter windows at 2Hz sampling.

Seeds. Each (session, method) combination is repeated for seeds $\{0, 1, 2\}$, varying model initialization and stochastic training elements; data splits (chronological 70/10/20 train/val/test) are fixed per session.

Training budget. DLinear/LSTM: 30 epochs. Time-LLM: 15 epochs. Batch size 16 throughout. All runs CPU-only (no GPU), frozen GPT-2 (124M) backbone for Time-LLM.

Orchestration. All 27 runs were driven sequentially by a resumable orchestration script, with each result checkpointed immediately so progress survives interruption. Aggregation (mean $\pm$ std per session/method) is computed from the raw per-run log.

4 Results

All 27/27 runs (3 sessions $\times$ 3 methods $\times$ 3 seeds) are complete.

4.1 Per-session results

Table 1: `profile_id=70`

Method	Test MSE	Test MAE	Train time (mean)
DLinear	0.0350 ± 0.0073	0.1330 ± 0.0247	0.05h
LSTM	0.0150 ± 0.0007	0.0629 ± 0.0041	1.58h
Time-LLM	0.0247 ± 0.0058	0.0985 ± 0.0218	8.13h

Each method wins exactly one of the three sessions on test MSE. On `profile_id=70`, LSTM beats Time-LLM by 31% MSE – a clean replication of the single-session pilot. On `profile_id=62`, all three methods sit within a very narrow, near-zero error band (this session’s `pm` target appears to have an unusually low dynamic range/noise floor, making the forecasting task close to saturated for all three methods regardless of capacity); DLinear is nominally lowest but the seed-to-seed spread

Table 2: `profile_id=62`

Method	Test MSE	Test MAE	Train time (mean)
DLinear	0.0003 $\pm$ 0.0000	0.0135 $\pm$ 0.0001	0.04h
LSTM	0.0007 $\pm$ 0.0005	0.0173 $\pm$ 0.0049	1.47h
Time-LLM	0.0004 $\pm$ 0.0002	0.0157 $\pm$ 0.0029	3.70h

Table 3: `profile_id=44`

Method	Test MSE	Test MAE	Train time (mean)
DLinear	0.0691 $\pm$ 0.0208	0.0985 $\pm$ 0.0049	0.05h
LSTM	0.0853 $\pm$ 0.0086	0.0940 $\pm$ 0.0020	0.53h
Time-LLM	0.0619 $\pm$ 0.0019	0.1236 $\pm$ 0.0247	6.58h

mostly overlaps with Time-LLM’s. On `profile_id=44`, **Time-LLM wins outright**: 27% lower MSE than LSTM and 10% lower than DLinear, with the smallest seed-to-seed std of the three methods on this session (± 0.0019 MSE) – the most confidently-won session in the whole matrix.

4.2 Cross-session summary

Table 4: Mean of per-session means, unweighted across sessions.

Method	Mean Test MSE	Mean Test MAE	Sessions won (MSE)
DLinear	0.0348	0.0817	1/3 (p62)
LSTM	0.0337	0.0581	1/3 (p70)
Time-LLM	0.0290	0.0793	1/3 (p44)

(See Section 5 for a caveat on why this specific unweighted aggregation should be read cautiously despite Time-LLM’s lead here.)

5 Discussion

No method dominates across sessions – the split verdict is the finding. Each of the three methods wins exactly one session on test MSE, and the cross-session mean and the cross-session “sessions won” count don’t even agree on a single winner by every metric: Time-LLM has the lowest mean MSE, but LSTM has the lowest mean MAE, and all three are tied 1-1-1 on session wins. This is the central methodological point of this study: a single-session comparison, however carefully run (the original pilot used a full 15/30-epoch budget, proper RevIN normalization, and a fair from-scratch baseline – nothing about the original comparison was sloppy) can still produce a method-ranking claim that reverses on a second, equally legitimate session of the same underlying real-world dataset. Multi-session evaluation is not a nice-to-have for honest claims about which forecasting approach “wins” on a sensor-data problem – per-session dynamics (noise floor, signal range, operating regime) can matter as much as, or more than, the modeling choice itself.

The cross-session mean MSE should be read with a specific caveat. `profile_id=62`’s errors are roughly two orders of magnitude smaller than the other two sessions’ (0.0003–0.0007 vs.

0.015–0.09), so an unweighted mean across sessions is dominated by whichever method happens to be marginally better on the two “harder” sessions (p70, p44) rather than reflecting a genuinely comparable average. Time-LLM’s cross-session MSE lead is real but should be understood as “wins the harder session (p44) outright, loses the other harder session (p70), ties on the easy one (p62)” rather than “wins on average” in any deeper sense – we report the unweighted mean because it is the simplest defensible aggregation given only 3 sessions, not because we think it is the last word on ranking.

Why might Time-LLM win specifically on profile_id=44? We do not have a confirmed mechanistic answer – this would require inspecting each session’s underlying operating conditions (load profile, thermal transient vs. steady-state behavior) in the source PMSM dataset, which is out of scope here. What we can say: `profile_id=44`’s std across seeds for Time-LLM (± 0.0019) is tighter than LSTM’s (± 0.0086) and DLinear’s (± 0.0208) on the same session, meaning Time-LLM’s advantage there is not an artifact of a single lucky seed. This is consistent with (though not proof of) the reprogrammed LLM’s frozen pretrained backbone providing more useful inductive bias on sessions with more complex/nonstationary dynamics than a small from-scratch model can pick up in 15–30 epochs on one session’s data alone.

Compute cost remains a first-class, session-independent result. Across all sessions, Time-LLM training took roughly $4\text{--}16\times$ longer than the LSTM baseline (1.5–8.1h vs. 0.5–1.6h) for accuracy that is session-dependently better, worse, or tied. This tradeoff is directly decision-relevant regardless of which method wins on accuracy for a given session, and matters even more once the result is “it depends which session you’re on” rather than a clean win for either side.

6 Limitations and Future Work

- **Backbone scale** is the most likely lever left untested: repeating these experiments with a 7B-parameter backbone (as in the original paper) on a GPU would directly test whether the session-split result changes with more model capacity.
- **Prompt-as-Prefix disabled** in this matrix for CPU-time reasons; a full-rigor PaP comparison (matched epochs, richer natural-language prompt) is a natural extension.
- **3 of 69 available sessions** were used; broader coverage (more sessions, or a formal stratified sample across operating conditions) would further strengthen the cross-session claim.
- **Time-LLM epoch budget (15) vs. LSTM/DLinear (30)** – asymmetric budgets are a possible confound worth noting, kept for CPU-time reasons.

7 Reproducibility

All code, exact hyperparameters, per-run logs, and the raw results file for every run reported here are available at:

<https://github.com/Jeevanmerkaji/Time-LLM>

References

Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by

- reprogramming large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations (ICLR)*, 2022.
- Wilhelm Kirchgässner, Oliver Wallscheid, and Joachim Böcker. Electric motor temperature dataset. <https://www.kaggle.com/datasets/wkirgsn/electric-motor-temperature>, 2021a. Paderborn University.
- Wilhelm Kirchgässner, Oliver Wallscheid, and Joachim Böcker. Data-driven permanent magnet temperature estimation in synchronous motors with supervised machine learning: A benchmark. *IEEE Transactions on Energy Conversion*, 36(3):2059–2067, 2021b.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.